Projet – MLSF: Un solveur SAT pour la résolution du jeu de réflexion Meowdoku – Partie I

Marc Aiguier - Pascale Le Gall - Romain Pascual

28 septembre 2026

Avant-propos

L'objectif général du projet est d'implanter un solveur SAT fondé sur l'algorithme DPLL [DP60, DLL62] (présenté en cours), de l'utiliser pour la résolution d'un jeu de réflexion et d'analyser ses performances. Le présent document décrit la partie I du projet décomposé en deux parties.

Le langage de programmation préconisé[1] est Python.

Le projet doit être réalisé en binôme[2]. Vous êtes invités à vous inscrire sur le fichier :
https://docs.google.com/spreadsheets/d/1KMkUxEgokC5iz8sjxAlg6CDLFCQmXKs_QMqIQBkUDhk/edit?usp=sharing

Rendu : Chacune des parties sera à rendre sur Edunao sous la forme d'un répertoire compressé comprenant

- un rapport rédigé décrivant pour chacune des questions les principales décisions de conception, les résultats et les analyses ;
- l'ensemble des fichiers (programmes, données, etc)

L'énoncé du projet contient des précisions sur les attendus du rendu du projet. Veuillez vous y conformer strictement. Cela permettra notamment de comparer automatiquement la performance de vos solveurs sur des données préparées par les enseignants.

Evaluation : Le projet sera évalué au regard des deux rapports déposés sur Edunao et d'une soutenance finale avec diapositives et démonstration. Vous êtes invités à mentionner, dans le rapport rédigé, les éléments que vous jugez pertinents, soit parce qu'ils sont originaux, soit parce que des questions n'ont été abordées que partiellement.

La date limite de dépôt de la partie I est[3] :

25 octobre 2026

Usage des assistants IA : leur emploi est *bien sûr* autorisé sous réserve qu'*in fine*, vous soyez capable de justifier et d'expliquer l'ensemble des éléments de vos rendus. En particulier, la soutenance finale donnera lieu à des questions précises à propos du code et du contenu du rapport. Vous êtes invités à mentionner, dans le rapport, l'usage que vous avez fait des assistants IA.

Le principal critère d'évaluation sera votre capacité à maîtriser, présenter et expliquer l'ensemble des éléments (algorithmes, code, évaluation, performances, choix de conception, analyse de complexité, ...) présents dans votre rendu. En clair, votre intérêt est de prioriser l'expertise à la quantité. Il est donc tout à fait possible que toutes les questions ne soient pas abordées, pour peu que celles qui sont réalisées le soient correctement. En d'autres termes, vous êtes invités à vous approprier vraiment les productions des assistants IA avant de les intégrer dans votre rendu afin de ne pas accumuler de *dette cognitive*[4].

[1] Ne pas choisir un autre langage de programmation sans en avoir discuté au préalable avec un enseignant.
[2] Avant de déroger à cette consigne, veuillez contacter un enseignant
[3] C'est une date limite ferme, avant la distribution de la Partie II du projet, prévue le 26 octobre.
[4] A ce sujet, vous pouvez feuilleter [KHY+25] sans en entreprendre la lecture.

Recommandation générale : bien lire l'énoncé jusqu'à la fin avant de commencer, car les questions ne sont pas indépendantes les unes des autres. Si vous êtes amenés à faire des aménagements au regard de l'énoncé, que ce soit pour le simplifier, le complexifier ou faciliter les analyses, veuillez l'indiquer dans le rapport, et si possible, argumenter votre proposition.

Contexte

Algorithme DPLL

Nous rappelons ci-dessous l'algorithme DPLL présenté en cours.

Algorithm 1: DPLL :satisfaisabilité

Input: Une formule propositionnelle F en forme normale conjonctive
Output: SAT si F satisfiable, UNSAT sinon

```
1  if F contient la clause vide □ then
2      return UNSAT
3  if F ne contient aucune clause then
4      return SAT
5  if F contient une clause unitaire {l} then
6      return DPLL(F[l ← 1])
7  if F contient un littéral pur l then
8      return DPLL(F[l ← 1])
9  choisir une variable p apparaissant dans F et une valeur de vérité b ;
10 if DPLL(F[p ← b]) then
11     return SAT
12 else
13     return DPLL(F[p ← b̄])
```

Dans l'algorithme 1, la notation $F[l \leftarrow b]$ avec l littéral (de la forme p ou $\neg p$) et b valeur de vérité (de la forme 1 ou 0, avec les convention $\overline{1} = 0$ et $\overline{0} = 1$) désigne la formule F simplifiée sous l'hypothèse que la valution de l vaut b.

En quelques mots, $F[l \leftarrow b]$ est $F[p \leftarrow b]$ si l s'écrit p, et $F[p \leftarrow \overline{b}]$ si l s'écrit $\neg p$. La formule simplifiée $F[p \leftarrow b]$ est définie en se ramenant à chacune des clauses C de F : si C contient le littéral p, $C[p \leftarrow 1]$ est la clause toujours vraie et est retirée de $F[p \leftarrow b]$; sinon $C[p \leftarrow 1]$ est la clause $C \setminus \{\neg p\}$. $C[p \leftarrow 0]$ est défini en suivant le même principle (selon que C contient ou non le littéral $\neg p$).

L'algorithme DPLL consiste à traiter successivement - et autant que possible - la propagation des clauses unitaires et l'élimination des littéraux purs. Lorsque la formule ne contient pas de clauses unitaires ou de littéraux purs, l'algorithme choisit aléatoirement la prochaine variable p à assigner, accompagnée d'une valeur de vérité b (ligne 9 définit la décision pilotant les branches **then** (ligne 10) et **else** (ligne 12)) .

L'algorithme DPLL peut être enrichi pour renvoyer non seulement le statut SAT / UNSAT (lignes 2, 4 et 11) de la formule en argument, mais aussi une valuation v, témoin de la satisfiabilité de la formule argument, si le statut est SAT. L'algorithme 2 est une adaptation de l'algorithme 1 pour renvoyer le statut SAT / UNSAT et une valuation v. Dans l'algorithme 2, les valuations sont représentées par un ensemble d'associations de la forme $p \leftarrow 1$ ou $p \leftarrow 0$.

Algorithm 2: DPLL : satisfiabilité et valuation

Input: Une formule propositionnelle F en forme normale conjonctive
Output: Un couple (s, v) où $s \in \{\text{SAT}, \text{UNSAT}\}$ et v est une valuation satisfaisante si $s = \text{SAT}$

if *F contient la clause vide* $\square$ **then**
  return $(\text{UNSAT}, 0)$;
if *F ne contient aucune clause* **then**
  return $(\text{SAT}, \emptyset)$;
if *F contient une clause unitaire* $\{l\}$ **then**
  $(s, v) \leftarrow$ `DPLL(`$F[l \leftarrow 1]$`)`;
  if $s = \text{SAT}$ **then**
    return $(\text{SAT}, v \cup \{l\})$;
  else
    return $(\text{UNSAT}, 0)$;
if *F contient un littéral pur l* **then**
  $(s, v) \leftarrow$ `DPLL(`$F[l \leftarrow 1]$`)`;
  if $s = \text{SAT}$ **then**
    return $(\text{SAT}, v \cup \{l\})$;
  else
    return $(\text{UNSAT}, 0)$;
$(p, b) \leftarrow$ `RandomChoice(`F`)` ;
$(s, v) \leftarrow$ `DPLL(`$F[p \leftarrow b]$`)`;
if $s = \text{SAT}$ **then**
  return $(\text{SAT}, v \cup \{p \leftarrow b\})$;
else
  $(s, v) \leftarrow$ `DPLL(`$F[p \leftarrow \overline{b}]$`)`;
  if $s = \text{SAT}$ **then**
    return $(\text{SAT}, v \cup \{p \leftarrow \overline{b}\})$;
  else
    return $(\text{UNSAT}, 0)$;
Fonction `RandomChoice(`F`)` **:**
  choisir aléatoirement une variable p apparaissant dans F;
  choisir aléatoirement $b \in \{1, 0\}$;
  return (p, b);

Format Dimacs

Le format Dimacs CNF est un format de fichier standard utilisé par les solveurs SAT pour encoder les problèmes de satisfiabilité booléenne sous forme normale conjonctive

Un fichier au format Dimacs respecte les règles suivantes :

- Commentaires : Toute ligne commençant par la lettre c est une ligne de commentaires, et donc ignorée.
- En-tête : La première ligne utile définit la taille du problème :

```
p cnf <nombre de variables> <nombre de clauses>
```

  Cette ligne est nécessaire.

- Chaque clause tient sur une ligne et est décrite par une suite d'entiers séparés par des espaces :

- Un entier positif k représente le littéral x_k de polarité positive.
- Un entier négatif $-k$ représente le littéral $\neg x_k$ de polarité négative.
- Chaque clause termine obligatoirement par la valeur 0 (marqueur de fin de clause).

Il n'y a pas d'ordre imposé, que ce soit pour l'ordre des clauses ou pour l'ordre des variables dans une clause.

Example 1 (L'énigme des 3 lampes) *Soit un système composé de 3 lampes : L_1, L_2 et L_3.*
On associe à chaque lampe $i \in \{1, 2, 3\}$ une variable booléenne x_i (x_i = Vrai si la lampe i est allumée). On impose les contraintes logiques suivantes :

- *Au moins une des 3 lampes doit être allumée :*

$$(x_1 \vee x_2 \vee x_3)$$

- *Si L_1 est allumée, alors L_2 doit être éteinte :*

$$(\neg x_1 \vee \neg x_2)$$

- *L_2 et L_3 ne peuvent pas être dans le même état :*

$$(\neg x_2 \vee \neg x_3) \wedge (x_2 \vee x_3)$$

La représentation du problème des lampes au format Dimacs est :
Variables : $x_1 \to$ 1, $x_2 \to$ 2, $x_3 \to$ 3 (3 variables au total).
Clauses : 4 clauses au total.

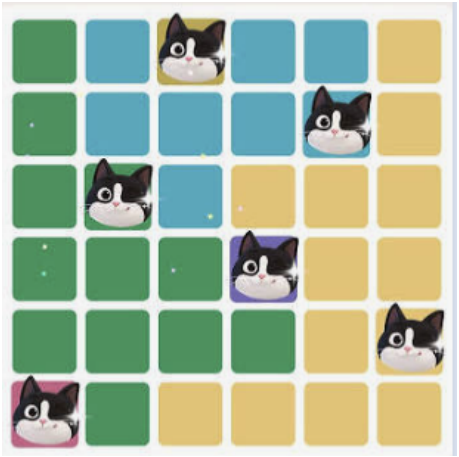
```
c --- Exemple : Enigme des 3 lampes ---
c Variables : 1 = L1, 2 = L2, 3 = L3
p cnf 3 4
1 2 3 0
-1 -2 0
-2 -3 0
2 3 0
```

Question 1

L'objectif de cette question est la modélisation du jeu de réflexion Meowdoku, qu'il est possible d'essayer sur la page https://www.meowdoku.org/play. Comme annoncé en préambule de cette partie, on vous propose dans cette section de formaliser le problème Meowdoku, un jeu de réflexion basé sur le placement de chats sur une grille découpée en régions colorées.

Pour une grille de taille $N \times M$, découpée en K régions colorées disjointes $R_1, R_2, \ldots, R_K$, le jeu consiste à placer K chats sur des cases distinctes (plus précisément, les régions $R_1, \ldots, R_K$ forment une partition de l'ensemble des cases de la grille : elles sont deux à deux disjointes, non vides, et leur union est l'ensemble des cases). Une configuration est valide si :

- Il y a au plus un chat par ligne.
- Il y a au plus un chat par colonne.
- Il y a exactement un chat par région colorée.
- Aucun chat n'est adjacent à un autre, ni horizontalement, ni verticalement, ni en diagonale (autrement dit, deux chats ne peuvent pas occuper deux cases adjacentes au sens du 8-voisinage).



(a) Exemple de grille Meowdoku remplie

```
c Grille 1
GRID 6 6
ZONES A B C D E F

A B C B B D
A B B B B D
A A B D D D
A A A E D D
A A A A D D
F A D D D D
```

(b) Fichier décrivant la grille

Figure 1: Exemple de grille Meowdoku et fichier correspondant

Le jeu est souvent présenté en ajoutant la contrainte supplémentaire $N = M = K$, ce qui revient à imposer un chat exactement par ligne et par colonne. La grille de la figure 1a a 6 lignes, 6 colonnes et 6 régions.

Le format imposé des fichiers des grilles Meowdoku est illustré par la figure 1b.

En accord avec l'intuition :

- Les lignes commençant par la lettre c sont des lignes de de commentaires, donc ignorées
- La ligne `GRID 6 6` décrit le format de la grille
- La ligne `ZONES A B C D E F` décrit le nombre et le nom des régions
- Les lignes

```
A B C B B D
A B B B B D
A A B D D D
A A A E D D
A A A A D D
F A D D D D
```

décrivent la répartition des régions sur la grille en mettant sur chaque case le nom de sa région.

Il est demandé d'implémenter un programme qui convertit une grille Meowdoku en un fichier Dimacs décrivant une formule CNF dont la satisfiabilité, et plus exactement la valuation témoin, fournit une solution au jeu décrit par la grille de départ.

Pour cela, il est demandé d'introduire une famille de variables $x_{i,j}$ pour $(i, j) \in \{1, \ldots, N\} \times \{1, \ldots, M\}$ et un ensemble de clauses portant sur ces variables tels qu'une solution de la grille consistera à placer un chat sur chacune des cases (i, j) telle que la variable $x_{i,j}$ associée est interprétée à vrai par une valuation témoin de l'ensemble de clauses en question.

Enfin, il est demandé d'écrire un programme qui génère des grilles de jeu Meowdoku, qui par construction, possèdent au moins une solution. Le programme en question sera paramétré par les dimensions (N, M et K) de la grille. A des fins de simplification, on peut se restreindre au cas $N = M = K$. Veuillez indiquer dans le rapport la méthode de construction de vos grilles de jeu.

```
c Grille 1
GRID 6 6
ZONES A B C D E F

A B C B B D
A B B B B D
A A B D D D
A A A E D D
A A A A D D
F A D D D D

A 3 2
B 2 5
C 1 3
D 5 6
E 4 4
F 6 1

. . * . . .
. . . . * .
. * . . . .
. . . * . .
. . . . . *
* . . . . .
```

Figure 2: Fichier décrivant une solution possible de la grille donnée en figure 1

Question 2

Il est demandé de

1. implémenter un programme qui lit un fichier Dimacs et le traduit en une structure de données Python pour les formules CNF
2. implémenter les deux versions 1 et 2 de l'algorithme DPLL.
3. valider les implémentations à l'aide des fichiers Dimacs fournis par les enseignants.
4. commenter les performances

Question 3

Il est demandé d'implémenter un programme qui étant donné un fichier grille tel que celui donné en figure 1b fournit une solution[5] sous le format donné par la figure 2. Le fichier résultat recopie le fichier fournit en entrée et décrit le résultat sous deux formats successivement : sous la forme d'un dictionnaire associant à chaque région, les coordonnées de la position de son chat, et d'une grille où les positions des chats sont marquées par le symbole $*$. Il conviendra de déléguer la recherche d'une solution à votre solveur en utilisant la conversion d'une grille de jeu en un fichier Dimacs. Il est conseillé de vérifier systématiquement la validité des solutions trouvées (en s'assurant que le fichier solution fournit une configuration licite pour la position des chats).

Question 4

En préambule, rappelons les éléments suivants (traduction libre d'un extrait de l'article [LL01]) :

[5] Par hypothèse, une solution existe si la grille fournie est licite.

Bien que la complexité dans le pire des cas de la procédure DPLL soit exponentielle, le temps d'exécution réel peut être fortement influencé par le choix des littéraux libres lors de l'étape de branchement. Il est possible de réduire de plusieurs ordres de grandeur la taille de l'arbre de recherche exploré par DPLL si les variables sont choisies dans le bon ordre. Bien que la recherche d'un ordre optimal soit difficile sur le plan théorique, de nombreux travaux de recherche ont été consacrés à l'élaboration de règles de branchement permettant de sélectionner des littéraux lors de l'étape de branchement de la DPLL. Elles fonctionnent assez bien en pratique par rapport à une stratégie aléatoire, mais n'offrent aucune garantie d'optimalité.

L'objectif de la présente question est d'explorer et comparer quelques heuristiques, notées de façon générique par la lettre H : l'algorithme 3 est une reformulation de l'algorithme 2 dans laquelle la fonction `RandomChoice` a été remplacée par la fonction `Heuristics` paramétrée par une heuristique en charge de sélectionner une variable propositionnelle et valeur de vérité susceptible de donner lieu à un retour en arrière (*backtracking*) si la branche **if** (ligne 23) échoue à statuer la satisfiabilité.

Algorithm 3: DPLL_H : satisfiabilité et valuation

Input: Une formule propositionnelle F en forme normale conjonctive et un choix d'heuristique H

Output: Un couple (s, v) où $s \in \{\text{SAT}, \text{UNSAT}\}$ et v est une valuation satisfaisante si $s = \text{SAT}$

```
1  if F contient la clause vide □ then
2      return (UNSAT, 0);
3  if F ne contient aucune clause then
4      return (SAT, ∅);
5  if F contient une clause unitaire {l} then
6      (s, v) ← DPLL(F[l ← 1]);
7      if s = SAT then
8          return (SAT, v ∪ {l});
9      else
10         return (UNSAT, 0);
11 if F contient un littéral pur l then
12     (s, v) ← DPLL(F[l ← 1]);
13     if s = SAT then
14         return (SAT, v ∪ {l});
15     else
16         return (UNSAT, 0);
17 (p, b) ← Heuristics(F,H) ;
18 (s, v) ← DPLL(F[p ← b]);
19 if s = SAT then
20     return (SAT, v ∪ {p ← b});
21 else
22     (s, v) ← DPLL(F[p ← b̄]);
23     if s = SAT then
24         return (SAT, v ∪ {p ← b̄});
25     else
26         return (UNSAT, 0);
27 Fonction Heuristics(F, H) :
28     choisir une variable p dans F et b ∈ {1, 0} selon l'heuristique H ;
29     return (p, b);
```

La définition des heuristiques nécessite quelques notations. Etant donnée une formule F en forme CNF :

- C dénote une clause générique de F, et on écrit alors $C \in F$ et $|C|$ pour désigner sa taille, i.e. le nombre de littéraux de C.

- p dénote une variable propositionnelle générique de F et on écrit alors $p \in F$ si p apparaît dans F. On note respectivement $\#(p, F)$, $\#^+(p, F)$ et $\#^-(p, F)$ le nombre d'occurrences de p dans F, le nombre d'occurrences du littéral p dans F et le nombre d'occurrences du littéral $\neg p$ dans F. On a alors les propriétés

$$(p \in F \Leftrightarrow \#(p, F) > 0) \text{ et } (\#(p, F) = \#^+(p, F) + \#^-(p, F)$$

  Les notations sont adaptées en fixant la taille des clauses. $\#_k(p, F)$, $\#_k^+(p, F)$ et $\#_k^-(p, F)$ le nombre d'occurrences de p dans les clauses de F de taille k, le nombre d'occurrences du littéral p dans les clauses de F de taille k et le nombre d'occurrences du littéral $\neg p$ dans les clauses de F de taille k. On a alors la propriété :

$$\#(p, F) = \sum_{k \in \mathbb{N}} \#_k(p, F)$$

- l dénote un littéral générique de F, de la forme p ou $\neg p$. On note $occ(l, F)$ le nombre d'occurrences de l dans F (si l s'écrit p (resp. $\neg p$), on a $occ(l, F) = \#^+(p, F)$ (resp. $occ(l, F) = \#^-(p, F)$).

  La notation est adaptée en fixant la taille des clauses. $occ_k(l, F)$ est le nombre d'occurrences de l dans les clauses de taille k de F.

Pour la formule F suivante :

$$(x_1 \vee x_2 \vee \neg x_4) \wedge (x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee x_4 \vee \neg x_5)$$

- F contient 3 clauses dont la clause $(x_1 \vee x_2 \vee \neg x_4)$ de taille 3
- $\#(x_2, F) = 3$, $\#^+(x_2, F) = 2$ et $\#^-(x_2, F) = 1$
- $\#_2(x_2, F) = 1$, $\#^+(x_2, F) = 2$ et $\#^-(x_2, F) = 1$
- Les littéraux de F sont $x_1, x_2, x_3, x_4, \neg x_2, \neg x_4$ et $\neg x_5$ avec à titre d'exemple $occ(x_4, F) = 1$.

A l'aide de ces notations, nous pouvons considérer les heuristiques :

H_0, ou *Choix Aléatoire (CA)*, correspond à l'heuristique mise en place dans l'algorithme 2. La variable de branchement, p, ainsi que sa valeur de vérité p sont choisies aléatoirement parmi respectivement les variables propositionnelles de F non encore affectées et les valeurs de vérité $\{1, 0\}$;

H_1, ou *Première Variable Libre (PVL)*, consiste à sélectionner systématiquement la première variable propositionnelle, non encore affectée, avec la valeur de vérité 1. Cette heuristique repose sur le fait que les variables propositionnelles de F sont munies d'un ordre naturel (comme c'est le cas pour le format Dimacs) ;

H_2, ou *Variable la plus Fréquente avec Polarité Majoritaire (VPPM)* consiste à choisir la variable propositionnelle p non affectée, maximisant $\#(p, F)$, et de lui affecter la valeur de vérité associée à la polarité majorité, i.e. 1 si $\#^+(p, F) \geq \#^-(p, F)$, et 0 sinon.

H_3, ou *Littéral le plus Fréquent (LP)* consiste à choisir le littéral l dont la variable n'a pas encore été affectée, maximisant $occ(l, F)$, et de lui affecter la valeur de vérité correspondante, i.e. 1 si l est de la forme p, et 0 si l est de la forme $\neg p$.

H_4, ou *Variable la plus Fréquente dans les Clauses de taille Minimale (VFCM)*[6] consiste à choisir une variable propositionnelle p, non encore affectée, parmi les plus présentes dans les clauses de taille minimale et de lui affecter la valeur de vérité majoritaire.

Pour $k = \min_{C \in F} |C|$, H_4 choisit parmi les variables non affectées une variable p qui maximise $\#_k(p, F)$, i.e. $p = arg \max_p \#_k(p, F))$

[6] MOMS en anglais pour Maximum Occurence in clauses of Minimum Size

Cette heuristique s'appuie sur l'intuition que les petites clauses sont critiques pour DPLL, car susceptibles de faire apparaître des clauses unitaires si elles sont réduites lors du choix de branchement.

La valeur de vérité choisie est la valeur de vérité majoritaire, i.e. 1 si $\#_k^+(p) \geq \#_k^-(p)$ et 0 sinon.

H_5, ou *Littéral le plus Fréquent dans les Clauses de taille Minimale (LFCM)* consiste à choisir le littéral l dont la variable n'a pas encore été affectée, maximisant $occ_k(l, F)$, et d'affecter à la variable associée p, la valeur de vérité correspondante, i.e. 1 si l est de la forme p, et 1 si l est de la forme $\neg p$.

Pour $k = \min_{C \in F} |C|$, H_5 choisit parmi les variables non affectées le littéral l qui maximise $occ_k(l, F)$, i.e. $l = arg \max_l occ_k(l, F))$

La valeur de vérité choisie est donnée par le littéral, i.e. 1 si l est de la forme p, 0 si l est de la forme $\neg p$.

H_6, ou *Variable de meilleure pondération (VJW)*[7] consiste à sélectionner la variable propositionnelle qui maximise

$$\sum_{k \in \mathbb{N}} \#_k(p, F) \times 2^{-k}$$

autrement dit, $p = arg \max_p \sum_{k \in \mathbb{N}} \#_k(p, F) \times 2^{-k}$

La valeur de vérité est alors 1 si

$$\sum_{k \in \mathbb{N}} \#_k^+(p, F) \times 2^{-k} \geq \sum_{k \in \mathbb{N}} \#_k^-(p, F) \times 2^{-k}$$

et 0 sinon.

A titre d'exemple, pour la formule donnée en exemple précédemment, on a

$$\sum_{k \in \mathbb{N}} \#_k(x_2, F) = \frac{1}{2^3} + \frac{1}{2^2} + \frac{1}{2^4} = \frac{7}{16}$$

Pour cet exemple, x_2 est la variable sélectionnée, avec la valeur de vérité 1

H_7, ou *Littéral de meilleure pondération (LJW)*[8] consiste à sélectionner le littéral qui maximise

$$\sum_{k \in \mathbb{N}} occ_k(l, F) \times 2^{-k}$$

autrement dit, $l = arg \max_l \sum_{k \in \mathbb{N}} \#_k(l, F) \times 2^{-k}$

La valeur de vérité est alors 1 si l est de polarité positive, et 0 sinon.

Les heuristiques H_2 à H_7 peuvent être déclinées en deux variantes, dites *statiques* et *dynamiques*, qui seront respectivement notées H_i^s et H_i^d pour $i \in 2..7$.
Elles sont

- statiques si la formule F référencée dans les énoncés des heuristiques fait référence à la formule initiale,
- et dynamiques si la formule F référencée dans les énoncés des heuristiques fait référence à la formule courante, c'est-à-dire, simplifiée et réduite par les étapes précédentes de l'application de DPLL (clause unitaire, polarité, branchement)

De façon générale,

[7] JW pour Jeroslow-Wang.
[8] JW pour Jeroslow-Wang.

- les versions statiques peuvent donner lieu à un prétraitement initial de la formule, ordonnant variables ou littéraux a priori, de sorte que sélectionner selon l'heuristique H_i^s consiste simplement à sélectionner la première variable non encore affectée fournie par ce prétraitement.
- les versions dynamiques nécessitent de mettre à jour ou de recalculer à chaque étape de branchement les valeurs d'intérêt ($\#(p, F)$, $occ(l, F)$ ou les sommes pondérées).

L'intérêt des versions dynamiques au regard des versions statiques repose alors sur la balance "coût de la mise à jour / pertinence de la sélection".

Il est demandé d'implémenter au moins 4 heuristiques différentes pour l'algorithme $DPLL_H$ avec les contraintes suivantes

- au moins une heuristique fondée sur la sélection d'un littéral (heuristiques H_3, H_5 ou H_6)
- au moins une heuristique mettant en jeu les tailles des clauses (H_4 à H_7)
- au moins une heuristique dynamique (H_i^d pour $i \in 2..7$)

Il est autorisé de définir vos propres heuristiques en combinant les heuristiques proposées (e.g. appliquer telle ou telle heuristique selon la taille de F ou le nombre de variables), ou en définissant vos propres critères.

Il est attendu que

- vous vous assuriez que vos différentes heuristiques donnent des résultats conformes à votre implémentation de l'algorithme 2 en particulier en les testant avec les données de test déjà mises en œuvre (Question 2 précédente) ;
- vous expliquiez les raisons qui ont conduit à vos choix d'heuristiques.

Question 5

Cette question a pour objectif d'instrumenter les solveurs DPLL_H implémentés à la Question 4 afin d'analyser et comparer leurs performances.

Les critères de comparaisons à considérer sont parmi les suivants :

1. nombre de décisions, i.e. le nombre de fois où la ligne 17 est exécutée
2. nombre de retours arrière (backtracks); i.e. le nombre de fois où la branche **else** (ligne 21) est exécutée;
3. nombre de propagation unitaire (ligne 5)
4. nombre de règle de littéral pur (ligne 11)
5. temps CPU
6. ratio de temps consacré au calcul des heuristiques
7. *autres critères de votre choix*

Il convient d'analyser les performances sur des instances très variées, selon la forme, selon le statut SAT/UNSAT, et issues de la Question 1 en lien avec le jeu de réflexion Meowdoku. En particulier, les résultats pourront être analysés séparément pour les instances SAT et les instances UNSAT.

Afin de consolider vos analyses, Vous êtes invités à considérer les questions suivantes :

- Les résultats observés sont-ils en adéquation ou en désaccord avec vos attentes ou intuitions ?
- En quelle mesure l'utilisation d'une heuristique améliore-t-elle la version initiale de DPLL (algorithme 2) ?
- Dans quelle mesure le recalcul dynamique améliore-t-il la qualité des heuristiques au regard de leur coût ?

Question 6

En cohérence avec les conclusions de la Question 5, vous êtes invités à affiner votre solveur optimisé pour le jeu de réflexion Meowdoku, qui sera mis en œuvre dans la compétition de solveurs interne à la mention.

Interface imposée du solveur

Afin de permettre une comparaison automatique des différentes implémentations, votre programme doit respecter une interface en ligne de commande commune.

Le point d'entrée imposé est un fichier `sat.py`. Il doit pouvoir être exécuté de la manière suivante :

```
python sat.py instance.cnf
```

Vous devez également fournir deux modes d'exécution explicites :

```
python sat.py instance.cnf --mode dpll
python sat.py instance.cnf --mode optimized
```

Le premier mode doit correspondre à l'implémentation de DPLL demandée en question 2. Le second mode doit correspondre à votre implémentation optimisée développée dans le cadre de cette question.

L'exécution sans mode explicite a simplement vocation à vérifier que votre programme s'exécute, et peut donc utiliser une implémentation ou l'autre, sans obligation.

Contraintes sur les deux modes. Les deux modes doivent :

- accepter un fichier au format DIMACS CNF en entrée ;
- produire un résultat correct pour toute instance valide ;
- utiliser exactement le même format de sortie ;
- retourner `SATISFIABLE` si la formule est satisfiable et `UNSATISFIABLE` sinon.

Les choix effectués pour l'implémentation optimisée sont libres. Vous pouvez notamment modifier les heuristiques de choix des variables, la stratégie de choix de polarité, la propagation, les structures de données ou toute autre partie de l'algorithme, à condition de préserver la correction du solveur.

Format de sortie. Pour une formule satisfiable, le programme doit produire :

```
s SATISFIABLE
```

Pour une formule insatisfiable :

```
s UNSATISFIABLE
```

Lorsque l'option `--output-model` est utilisée et que la formule est satisfiable, le programme doit également produire un modèle :

```
python sat.py instance.cnf --output-model
```

Par exemple :

```
s SATISFIABLE
v 1 -2 3 -4 5 0
```

Un littéral positif indique que la variable correspondante vaut `True`, tandis qu'un littéral négatif indique qu'elle vaut `False`. La ligne représentant le modèle doit se terminer par `0`.

Aucun modèle ne doit être produit pour une formule insatisfiable.

Sortie standard. Le résultat du solveur doit être écrit sur la sortie standard (`stdout`). Cette sortie pourra être redirigée vers un fichier par l'infrastructure de comparaison, par exemple :

```
python sat.py instance.cnf > result.txt
```

La sortie standard est réservée au résultat du solveur. Votre programme ne doit donc pas y afficher de messages de débogage, de progression ou de mesures de temps. Si vous avez besoin de tels affichages pendant le développement, utilisez la sortie d'erreur (`stderr`) ou désactivez-les dans la version finale.

Correction du modèle. Lorsqu'un modèle est fourni, celui-ci doit effectivement satisfaire l'ensemble des clauses de l'instance DIMACS.

Les deux modes `dpll` et `optimized` doivent naturellement retourner le même statut logique pour une instance donnée. Le modèle retourné peut cependant être différent.

Environnement d'exécution Les performances des solveurs seront évaluées dans un environnement commun afin de rendre les mesures comparables entre les groupes.

La compétition sera exécutée sur la machine suivante :

- **Processeur :** Intel(R) Core(TM) Ultra 7 155H ;
- **Architecture :** x86_64 ;
- **Cœurs physiques :** 16 ;
- **CPU logiques :** 22 ;
- **Mémoire vive :** 30 GiB ;
- **Système d'exploitation :** Ubuntu 22.04 ;
- **Noyau Linux :** 6.8.0-138-generic.

Les solveurs seront exécutés dans un environnement Docker commun, utilisant les versions suivantes :

- **Docker :** 29.8.1 ;
- **Docker Compose :** 5.5.1 ;
- **Python :** 3.12 ;
- **Image de base :** `python:3.12-slim`.

Afin de garantir des conditions d'exécution comparables, chaque exécution est limitée à :

- **4 CPU** au maximum ;
- **8 GiB de mémoire vive** au maximum ;
- **60 secondes par instance**.

Le temps mesuré correspond au temps nécessaire à l'exécution du solveur et à la production de son résultat. Il pourra être allongé sur certaines instances.

Classement Le classement des groupes est établi selon les critères suivants, dans cet ordre :

1. Le nombre d'instances correctement résolues, par ordre décroissant.
2. En cas d'égalité, le temps total de résolution sur les instances correctement résolues, par ordre croissant.

Une instance est considérée comme *correctement résolue* si:

- la formule est insatisfiable et la réponse fournie par le solveur est `UNSATISFIABLE` ;
- la formule est satisfiable, le solveur répond `SATISFIABLE` et le modèle retourné satisfait effectivement la formule.

Dans ce deuxième cas, le modèle fourni est vérifié indépendamment du solveur : toutes les clauses de la formule doivent être satisfaites par l'affectation proposée. Toute réponse incorrecte (mauvais statut SAT/UNSAT ou modèle non satisfaisant) rend l'instance incorrectement résolue. Elle ne contribue alors ni au nombre d'instances correctement résolues, ni au temps total du groupe.

Extra bonus à la clé : le binôme qui livrera le meilleur solveur bénéficiera directement de la note 20 pour le projet.

References

[DLL62] Martin Davis, George W. Logemmann, and Donald W. Loveland. A machine program for theorem-proving. *Communications of th ACM*, 5:394–397, 1962.

[DP60] Martin Davis and Hilary Putnam. A machine program for theorem-proving. *Communications of th ACM*, 7(3):201–215, 1960.

[KHY+25] Nataliya Kosmyna, Eugene Hauptmann, Ye Tong Yuan, Jessica Situ, Xian-Hao Liao, Ashly Vivian Beresnitzky, Iris Braunstein, and Pattie Maes. Your brain on chatgpt: Accumulation of cognitive debt when using an ai assistant for essay writing task, 2025.

[LL01] Michail G. Lagoudakis and Michael L. Littman. Learning to select branching rules in the dpll procedure for satisfiability. *Electronic Notes in Discrete Mathematics*, 9:344–359, 2001.